

## 1.2 Generate Assembly

Compile the file `ArrayAccu.c` to RISC-V assembly code using:

```
riscv32-unknown-elf-gcc -O1 -S ArrayAccu.c -o ArrayAccu.s
```

The switch `-S` tells `gcc` to produce human-readable assembly code instead of a machine-readable binary. The switch `-o` tells `gcc` to store the output in `ArrayAccu.s`. Finally, `-O1` (capital letter O) enables level-one optimization. This lab compares `-O0`, which disables optimization, with `-O1`.

Look at the content of `ArrayAccu.s` and answer the following questions:

1. The program starts running with the C function `main`. Where is `main` defined in `ArrayAccu.s`?
2. Search through `ArrayAccu.s`, where is `main` being called?
3. The `ArrayAccu()` function is called within `main`. Look at the code before the function is called and the code **inside** `ArrayAccu()`. Where are the input arguments and return value stored?
4. Can you see how the return address `ra` is used?

1. The program starts running with the C function `main`. Where is `main` defined in `ArrayAccu.s`?

The main label is the entry address of the assembly code

2. Search through `ArrayAccu.s`, where is `main` being called?

The program will run `main` when it starts up.

3. The `ArrayAccu()` function is called within `main`. Look at the code before the function is called and the code **inside** `ArrayAccu()`. Where are the input arguments and return value stored?

Input arguments store in stack, start address store in `a0`

Return value store in `a0`

4. Can you see how the return address `ra` is used?

First `ra` store the return address of the main program.

and we store it into stack, call `ArrayAccu`, `ra` became the return address from `ArrayAccu` to `main`.

When return back to `main`, we set `PC = ra`, and return to `main`.

and we restore `main` return address to `ra` from stack, and back to `main` caller.

**1.4 Branches** Compile `branch1.c` and `branch2.c` at optimization levels `-O0` and `-O1`:

```
for name in branch1 branch2; do
    riscv32-unknown-elf-gcc -O0 -S ${name}.c -o ${name}_0.s
    riscv32-unknown-elf-gcc -O1 -S ${name}.c -o ${name}_1.s
done
```

Read through the compiled code and answer the following questions:

1. Does the compiler implement the two branches differently? How are they different?
2. How many static RISC-V instructions are present in each function? Do not count labels or assembler directives.
3. How does the generated code differ between `-O0` and `-O1`?

1. Does the compiler implement the two branches differently? How are they different?

No, because the .c program is different already  
, so compiler implement based on the differences in .c

2. How many static RISC-V instructions are present in each function? Do not count labels or assembler directives.

branch1-0.s: 22

branch1-1.s: 5

branch2-0.s: 22

branch2-1.s: 5

3. How does the generated code differ between `-O0` and `-O1`?

-O0 follow exactly what .c file says

-O1 keeps the necessary code, minimize instruction

**1.5 Loops** Compile loop1.c and loop2.c using -O0 and -O1, similar to the preceding exercise. Then answer the following questions:

1. How does the compiler implement the two different loops?
2. How many instructions are needed for the two loops?
3. How does the generated code differ between -O0 and -O1?
4. What work does loop fusion remove, and why can the compiler no longer delete the loops entirely?

1. How does the compiler implement the two different loops?

For Loop1, the compiler write 2 loops  
but in Loop2, the compiler write all thing in 1 loop.  
Same for -O0 and -O1 version

2. How many instructions are needed for the two loops?

|                | Static | Dynamic:                                           |
|----------------|--------|----------------------------------------------------|
| For Loop1-O.S: | 60     | $12 + 22 \times 5 + 2 + 3 + 22 \times 5 + 5 = 242$ |
| For Loop1-I.S: | 23     | $2 + 9 \times 5 + 2 + 9 \times 5 + 1 = 95$         |
| For Loop2-O.S: | 52     | $12 + 38 \times 5 + 5 = 207$                       |
| For Loop2-I.S: | 16     | $2 + 13 \times 5 + 1 = 68$                         |

3. How does the generated code differ between -O0 and -O1?

-O0 follow exactly what .c file says  
-O1 keeps the necessary code, minimize instruction

4. What work does loop fusion remove, and why can the compiler no longer delete the loops entirely?

Remove a extract loop by combining two loops together. We only need to initialize int i once, and only need one branch.

Because 1 loop is needed to complete the task. Fully unroll can remove all the loop, but it is not decrease the computations and register manipulation.

**1.6 If-then-else vs. Switch** Compile `branch.c` and `case.c` using `-O0` and `-O1` as before, and answer the following questions:

1. How does the compiler implement the following two code segments differently?
2. How does the generated code differ between `-O0` and `-O1`?

1. How does the compiler implement the following two code segments differently?

In `-O0` version, `branch` will branch to another label if the condition is not satisfied, but `case` will find out the condition and then jump to the execute instruction

In `-O1` version `branch` and `case` are the same.

2. How does the generated code differ between `-O0` and `-O1`?

`-O0` use will check if the condition is satisfied or not and then calculate the result

But `-O1` will calculate the result, if condition is satisfied, then return the result.

**1.7 Function Calls** Compile and compare `funca11.c` and `funca12.c` using `-O0` and `-O1`, then answer the following questions:

1. How does the compiler implement the following two code segments differently?
2. How does the generated code differ between `-O0` and `-O1`?

1. How does the compiler implement the following two code segments differently?

In `-O0` version, `funca1` pass the value of `a` and `b` into `fun` function and do the calculation directly.

But `funca2` pass the address into `fun` function

In `-O1` version, both 1 and 2 ignore `fun`, they call `--addtf3` directly

2. How does the generated code differ between `-O0` and `-O1`?

`-O0` need to either pass the value or address to `fun` function to do the calculation.

But `-O1` directly use `--addtf3` and `--fixdfsi` itself and ignore `fun`.